Template Provenance: Prompt-Template Sandbox Escapes as a Systematic Weakness Class Across Agent Frameworks

Abstract

Server-side template injection (SSTI) is a well-understood web vulnerability, yet the same primitive keeps reappearing inside AI agent frameworks and model serving stacks—not in web pages, but in the prompt templates that sit between user configuration and model invocation. We systematize seven publicly disclosed incidents from 2024–2026 (RAGFlow CVE-2026-28797, GitLab AI Gateway CVE-2026-1868 and CVE-2026-90970, SGLang CVE-2026-5760, llama-cpp-python CVE-2024-34359, Giskard Agents CVE-2026-34172, vLLM CVE-2025-61620) under a new *provenance-based* taxonomy: who supplies the template (operator prompt, user-authored flow configuration, model-file metadata, or API-client request parameters) crossed with the sandbox posture of the render path (unsandboxed, Jinja2-sandboxed, or no template engine at all). We demonstrate the taxonomy's posture column with a differential-testing harness that replays a corpus of 10 documented Jinja2 escape and sandbox-bypass payloads through reconstructed render-posture cores mirroring the Jinja2 constructor choices of eight pinned framework render paths (80 render cells). Each framework-to-posture mapping is grounded in a cited source line (file and version or commit) in §4; the harness shows the reconstructed postures produce the stated outcomes (it does not re-test the frameworks' own code). Unsandboxed paths execute all 8 escape-class payloads (8/8); sandboxed paths block all of them (0/8); the reconstructed postures of the shipped fixes in RAGFlow and SGLang block the full set. Our version-history analysis shows the weakness recurs at re-implementation boundaries: Hugging Face `transformers` has sandboxed chat-template rendering since at least October 2023, yet SGLang and llama-cpp-python independently re-implemented rendering with a raw `jinja2.Environment`, and GitLab's gateway broke twice in eight months in the same component class. We further show that sandboxing is necessary but not sufficient: vLLM's CVE-2025-61620 achieves denial of service through attacker-controlled template *selection* even where rendering itself is sandboxed. We close with defensive guidance for framework authors and operators. All testing was local-only, used non-destructive payloads against historical pinned versions, and found no new live vulnerabilities.

1 Introduction

Modern AI systems are stitched together with string templates. Agent frameworks let users author multi-step workflows whose prompts are assembled at runtime; model servers render chat templates shipped inside model files; API gateways expand flow configurations into prompts before invoking a language model. In the Python ecosystem, the engine behind most of this rendering is Jinja2—a template language expressive enough to reach arbitrary Python objects, and therefore dangerous when the *template itself* is attacker-controlled. The resulting flaw is server-side template injection (SSTI), cataloged as CWE-1336 (improper neutralization of special elements used in a template engine).

In the last two years, this flaw class has produced a striking run of critical CVEs in AI infrastructure: a 9.7 in llama-cpp-python via a poisoned GGUF model file [6], a 9.8 in SGLang's rerank endpoint via the same vector [4], an 8.7/8.8 (CVSS v4.0/v3.1) in RAGFlow's agent workflow components [1], an 8.8 (CVSS v3.1) in Giskard Agents' `ChatWorkflow` [5], and two CVSS-9.9 flaws in GitLab's AI Gateway eight months apart [2, 3]. Each was reported and patched in isolation. No published work systematizes them.

This paper makes three contributions. **First**, a provenance-based taxonomy of template injection in AI/agent infrastructure. Existing SSTI taxonomies are engine-centric (which engine, which payload). Ours is *supply-chain-centric*: the rows are who supplies the template, because that determines the attacker's cost and the defender's leverage. **Second**, a differential demonstration: a harness that replays 10 documented escape payloads through reconstructed render-posture cores for eight framework paths at pinned versions, showing the taxonomy's posture column determines exploitability of documented gadgets. The framework-to-posture mapping is carried by cited constructor lines, not asserted. **Third**, structural

findings from version-history analysis—where the sandboxes were added, where they were missed, and why the same bug recurs.

We deliberately scope to measurement, not exploitation: all payloads are non-destructive (read-only), all vulnerable versions are historical and already patched, and our sweep of current code found no new live flaws (Section A).

2 Background

Server-side template injection. When an application embeds user input into a template *string* that is then compiled and rendered, the input can break out of data context into code context. PortSwigger's SSTI methodology [11] documents the canonical detection probe (`{{7*7}}` rendering as `49`) and the escalation ladder: attribute traversal (`".__class__`), object-graph navigation to `os.popen` or `subprocess.Popen`, and finally arbitrary command execution. The methodology is practitioner-oriented; it is not a measurement study and does not address AI infrastructure.

Jinja2 sandboxing. Jinja2 ships an opt-in sandbox (`jinja2.sandbox.SandboxedEnvironment` and the immutable variant) that intercepts attribute access and blocks dunder attributes (`__class__`, `__globals__`, ...), defeating the documented escape gadgets. The sandbox is explicitly best-effort: Jinja2's own documentation warns against rendering untrusted templates even in the sandbox [13]. Critically, the *default* `jinja2.Environment` and `jinja2.Template` perform no such interception—every render is a remote-code-execution primitive if the template string is attacker-controlled.

Where templates live in AI stacks. Three roles recur. *Agent frameworks* (RAGFlow, LangChain/LangGraph, GitLab Duo Agent Platform) compose prompts from operator templates, user-authored workflow configurations, and runtime variables. *Model servers* (vLLM, SGLang, llama-cpp-python) render *chat templates*: Jinja2 programs shipped as model metadata (`tokenizer.chat_template`) that format message lists before inference. *API gateways* (GitLab AI Gateway, vLLM's OpenAI-compatible server) accept template-related parameters from clients. Each role places a template compiler on a trust boundary; the question is always who controls the template string and whether a sandbox stands between it and the interpreter.

3 A Provenance-Based Taxonomy

Engine-centric taxonomies ask *which* template engine is in play. For AI infrastructure, the sharper question is *who supplies the template string*, because provenance determines both attacker cost (writing a flow config vs. poisoning a model file vs. sending an API parameter) and the available defenses (authentication, model signing, parameter allowlists). Table 1 crosses four provenance classes with three sandbox postures observed in the wild.

Table 1: Provenance × sandbox-posture taxonomy. Each cell lists a measured, incident-mapped, or source-cited instance (§§4–5); U = unsandboxed, S = Jinja2 sandbox, N = no template engine.

Template provenance	U	
T1 operator prompt template	—	LangChain
T2 user-authored flow config	RAGFlow ≤0.24.0 Giskard ≤0.3.3	GitLab RAGFlow
T3 model-file metadata	SGLang ≤0.5.9 llama-cpp-python <0.2.72	SGLang HF tran
T4 API-client template params	—	vL

*GitLab's gateway *had* a sandbox; CVE-2026-90970 escaped it—posture S, outcome escape. ‡GitLab is advisory-mapped only (closed source; posture taken from the advisory text). †vLLM's rendering was sandboxed (via HF transformers) yet CVE-2025-61620 achieves DoS through template *selection*; see §6. RAGFlow, SGLang, llama-cpp-python, Giskard, LangChain, and HF transformers postures are demonstrated in the harness (paths A–H, §4); Haystack is source-cited (`haystack/utils/jinja2_sandbox.py`).

T1: operator prompt template. The template is authored by the application developer; only *data* variables are user-controlled. This is LangChain's `jinja2_formatter` default use. Risk is low absent a second-order injection (user data that is itself later treated as a template).

T2: user-authored flow configuration. The template string is authored by a user of the platform—in RAGFlow, the author of an agent workflow's StringTransform/Message component; in GitLab, the author of a Duo Agent Platform custom flow. The author is authenticated but often low-privilege (any developer with flow access), so this is a privilege-boundary crossing, not merely a trust issue.

T3: model-file metadata. The template arrives inside a model artifact (`tokenizer.chat_template` in GGUF/Hugging Face models). The victim is whoever *loads* the model—a supply-chain attack. The attacker pays the one-time cost of publishing a poisoned model; every deployment that loads it is compromised when the template renders.

T4: API-client template parameters. The client supplies the template (or its parameters) per request—vLLM's `chat_template` and `chat_template_kwargs`. Here the novel sub-pattern is *selection* vs. *escape*: CVE-2025-61620

does not break the sandbox; it replaces *which* template renders (via a `dict.update` kwargs override) and chooses a resource-exhausting one.

Sandbox postures. **U**—raw `jinja2.Template` or `jinja2.Environment`: every documented gadget executes. **S**—`SandboxedEnvironment`/`ImmutableSandboxedEnvironment`: documented dunder gadgets are blocked. **N**—no template engine on the path (Go rewrite, `f-string`/mustache): the SSTI primitive does not exist.

Two cells deserve emphasis because they break the naive "sandbox = safe" intuition. GitLab's CVE-2026-90970 is an escape *of* a sandbox (posture S, outcome escape): the advisory states the attacker "escape[s] the prompt template sandbox via a specially crafted flow configuration" [2]. And vLLM's CVE-2025-61620 (CWE-400, not CWE-1336) shows that even correct sandboxing leaves a resource-exhaustion channel when the attacker controls template selection.

4 Methodology

Falsifiable question. For each framework render path: (1) does user-controlled input reach a template engine, (2) is a sandbox present, and (3) does a known escape payload execute? The harness below demonstrates that the taxonomy's posture column determines the outcomes; the framework-to-posture mapping itself rests on the cited constructor lines (Table 2), not on the harness alone.

Corpus. Ten payloads drawn from documented SSTI gadgets (PortSwigger [11], Jinja2 sandbox docs [13]): P1 engine-detection (`{{7*7}}`); P2 dunder attribute read; P3–P4 OS command execution via `cycler/namespace __init__.__globals__.os.popen` and P5 via `lipsum.__globals__.os.popen` (read-only: each cats a local canary file whose content we check in the rendered output); P6 an `attr()`-filter obfuscation variant; B1–B3 sandbox-bypass variants (tuple/dict dunder traversal, attr-filter chain); B4 a benign control (`namespace(x=1).x`) that must render even in a sandbox.

Render paths. Eight reconstructed render-posture cores, each mirroring the Jinja2 constructor choice of a pinned framework source listed in Table 2. The cores are faithful to the constructor and its sandbox posture (verified against the pinned sources and the cited advisories) but are reconstructions, not copies of framework code: the harness demonstrates that the claimed postures produce the claimed outcomes rather than independently measuring the frameworks. Pins are pinned vulnerable versions and current HEAD (full pins in Appendix D). Each payload is rendered with empty/minimal data bindings; outcomes are classified as `engine-eval`, `escaped`, or `blocked`. A separate logic-level replay tests vLLM's kwargs-override vector (old `dict.update` merge vs. fixed `resolve_chat_template_kwargs`).

Table 2: Render-posture cores: the pinned source and constructor choice each core mirrors. PR merge dates and diffs verified via the GitHub API, 2026-10-09.

Path	Framework source @ pin	Constructor choice
A	RAGFlow v0.24.0, `string_transform.py:99`	`jinja2.Template(s`
B	LangChain HEAD, `prompts/string.py:72`	`SandboxedEnvironm`
C	SGLang v0.5.9, `serving_rerank.py:128`	`jinja2.Environmen`
D	SGLang HEAD, `serving_rerank.py:131`	`ImmutableSandboxe`
E	RAGFlow post-#13305[a]	module-level `Sandbox`
F	HF transformers HEAD, `chat_template_utils.py:495`	`ImmutableSandboxe`
G	llama-cpp-python <0.2.72, `llama_chat_format.py`[b]	`jinja2.Environmen`
H	Giskard ≤0.3.3, `templates/environment.py:37`[c]	`Environment(autoe`

[a] https://github.com/infiniflow/ragflow/pull/13305: "security: Adopt Jinja2 SandboxedEnvironment for template rendering," merged 2026-03-02.
[b] https://github.com/advisories/GHSA-56xg-wfcc-g829: `Jinja2ChatFormatter.__init__` builds `jinja2.Environment(loader=jinja2.BaseLoader(), trim_blocks=True, lstrip_blocks=True).from_string(self.template)` with no sandbox.
[c] https://github.com/advisories/GHSA-frv4-x25r-588m: `_inline_env = Environment(autoescape=False)` ("Not SandboxedEnvironment"), used by `MessageTemplate.render()`.

Table 3: Differential-testing outcomes. Escape-class payloads = P2–P6, B1–B3 (8 payloads). P1 = engine detection; B4 = benign control.

Path	Posture	P1	Escape-class	B4
A RAGFlow v0.24.0	U	engine-eval	**8/8 escaped**	engine-eval
B LangChain	S	engine-eval	0/8 blocked	engine-eval
C SGLang v0.5.9	U	engine-eval	**8/8 escaped**	engine-eval
D SGLang HEAD	S	engine-eval	0/8 blocked	engine-eval
E RAGFlow #13305	S	engine-eval	0/8 blocked	engine-eval
F HF transformers	S	engine-eval	0/8 blocked	engine-eval
G llama-cpp-python <0.2.72	U	engine-eval	**8/8 escaped**	engine-eval
H Giskard ≤0.3.3	U	engine-eval	**8/8 escaped**	engine-eval

Ethics and scope. All runs are local and CPU-only. Payloads are non-destructive (read-only canary reads; no writes, no network, no exfiltration). Vulnerable versions exercised are historical, publicly documented, and patched. No live services were probed, no third-party exploit code was used, and our sweep of current framework code found no new live flaws, so no vendor disclosure was triggered (disclosure log in the artifact appendix; ethics discussion in Appendix A).

5 Measurement Results

Table 3 summarizes the 80 render cells (8 paths × 10 payloads). The differential is stark and uniform.

Unsandboxed paths execute everything. On paths A, C, G, and H, all four RCE-class payloads (P3–P6) exfiltrated the canary file content through `os.popen`, and all three bypass variants (B1–B3) succeeded. The RAGFlow path additionally reproduces the same gadget chain as CVE-2026-28797: `Jinja2Template(script)` over a user-supplied `script`, with render exceptions swallowed (`except Exception: pass`), which also means failed probes are silent. Paths G and H confirm the two newly mapped incidents behave as their advisories state: the llama-cpp-python posture (raw `jinja2.Environment` over a model-supplied chat template) and the Giskard posture (raw `Environment` over a developer-supplied message) each execute 8/8 escape-class payloads.

Sandboxed paths block everything malicious, allow the benign. On paths B, D, E, F, every escape-class payload raised `SecurityError` (dunder access denied) or rendered inert; the benign control B4 rendered correctly on all eight paths, confirming the sandboxes discriminate rather than merely disabling templating. Path E verifies RAGFlow's shipped fix (#13305, 2026-03-02)[1]: the identical payload set that escaped at v0.24.0 is fully blocked after the change to `SandboxedEnvironment().from_string`. Note that on path E "blocked" means silent non-render: the sandbox raises `SecurityError`, which the RAGFlow wrapper swallows (`except Exception: pass`), so the raw template is returned unrendered rather than an error being surfaced.

vLLM kwargs override (logic replay). The pre-fix merge (`_chat_template_kwargs.update(chat_template_kwargs)`) resolves to the attacker-supplied template whenever the client smuggles a `chat_template` key inside `chat_template_kwargs`—an independent analysis notes that "simply forbidding the `chat_template` parameter does not fully mitigate" [10]. The fixed `resolve_chat_template_kwargs` raises `ValueError` on the smuggled key. This is a *selection* primitive, distinct from sandbox escape, and it is why CVE-2025-61620 is CWE-400 (resource exhaustion) rather than CWE-1336.

Per-incident mapping. Table 4 places the seven public incidents in the taxonomy. Inclusion criterion: a publicly disclosed CVE/GHSA (2024–2026) in AI/agent infrastructure whose root cause is a template engine compiling an attacker-influenced template string. Two corrections to the informal record are worth making explicit: (i) CVE-2025-61620 is classified CWE-400, not CWE-1336—it belongs in this paper because the weapon is a template, but the weakness is uncontrolled resource consumption; (ii) the AWS Loom CVEs sometimes mentioned alongside this cluster (auth bypass / OAuth2 / SSRF) are not template escapes and are excluded.

[1] https://github.com/infiniflow/ragflow/pull/13305

Table 4: Public incidents mapped into the taxonomy. CVSS as published. "U/S" for CVE-2026-1868: the advisory describes insecure template expansion; the pre-fix posture could not be confirmed as sandboxed, so we list both.

CVE	Framework	Prov.	Posture	CVSS
2026-28797	RAGFlow $\leq$0.24.0	T2	U	8.7 (v4.0)
2026-90970	GitLab AI Gateway	T2	S$\rightarrow$esc.	9.9
2026-1868	GitLab AI Gateway	T2	U/S	9.9
2026-5760	SGLang $\leq$0.5.9	T3	U	9.8
2024-34359	llama-cpp-python $<$0.2.72	T3	U	9.7
2026-34172	Giskard Agents $\leq$0.3.3	T2	U	8.8 (v3.1)
2025-61620	vLLM	T4	S (DoS)	6.5

CVE-2026-34172 (Giskard Agents, GHSA-frv4-x25r-588m, listed in the OWASP GenAI LLM Top 10 2026 references) meets the criterion and is included.

6 Structural Findings

F1: The weakness recurs at re-implementation boundaries. Hugging Face `transformers` has rendered chat templates inside `ImmutableSandboxedEnvironment` since at least October 2023—the earliest `apply_chat_template` revision we checked (commit `8b46c5bc`, 2023-10-04) already compiles templates with `ImmutableSandboxedEnvironment(trim_blocks=True, lstrip_blocks=True)`; the dedicated `chat_template_utils` module (created in PR #30621 on June 11, 2024[2]) inherited the same constructor when the render helper moved there (commit `371b9c14`, 2024-08-23)—the safe default existed from day one. Yet two downstream projects re-implemented template rendering from scratch and reintroduced unsandboxed rendering: llama-cpp-python (model-supplied chat templates rendered without sandboxing; fixed in 0.2.72, May 2024) and SGLang's rerank path (feature added 2026-01-14 in #16403[3] with a raw `jinja2.Environment()`; fixed 2026-04-29 in #23660[4], a +3/–2-line diff). The vulnerability window in SGLang was about 105 days. The lesson is architectural: every hand-rolled render path is a new chance to forget the sandbox. Frameworks should route all template rendering through a single hardened choke point (as HF transformers does) rather than letting each endpoint grow its own.

F2: Point fixes invite repeat findings. GitLab's AI Gateway suffered two CVSS-9.9 template-engine flaws in the same

[2] https://github.com/huggingface/transformers/pull/30621: "Chat Template support for function calling and RAG," merged 2024-06-11.

[3] https://github.com/sgl-project/sglang/pull/16403: "feat: support qwen3(-VL) rerank scoring&chat template," merged 2026-01-14.

[4] https://github.com/sgl-project/sglang/pull/23660: "Fix for CVE-2026-5760," merged 2026-04-29, +3/–2 lines.

component class eight months apart: CVE-2026-1868 (insecure template expansion via crafted flow definitions, patched Feb 6, 2026 in 18.6.2/18.7.1/18.8.1) and CVE-2026-90970 (prompt-template sandbox escape via crafted flow configuration, patched Oct 2, 2026 in 19.2.4/19.3.2/19.4.1). Notably, the October advisory does not reference the February one. When the remediation is "fix this escape" rather than "remove attacker-controlled templates from this trust boundary," the next escape is a matter of time. RAGFlow's trajectory is the contrast: after sandboxing the Python components (#13305[5]), the project eliminated the render surface by rewriting the agent in Go (Python sources removed 2026-09-28, #20289[6]).

F3: Sandbox $\neq$ resource safety. CVE-2025-61620 is the counterexample to "just add a sandbox." vLLM's rendering delegates to HF transformers (sandboxed), yet an authenticated client could still deny service to the server by controlling *which* template renders—either directly via the `chat_template` parameter or by smuggling it inside `chat_template_kwargs` through `dict.update`. The fix (PR #25794, merged 2025-09-27[7]) has two parts: `resolve_chat_template_kwargs` rejects unexpected keys (`chat_template`, `tokenize`) and resolves remaining kwargs against the template's declared variables, and `safe_apply_chat_template` wraps rendering. The general principle: sandbox the *engine*, allowlist the *template source*, and bound the *resources* (timeouts, output caps)—three independent controls.

F4: The fixes are small; the detection is the hard part. RAGFlow's fix was a two-file, one-idiom change (`Jinja2Template(x)` $\rightarrow$ `_jinja2_sandbox.from_string(x)`); SGLang's was +3/–2 lines. In both cases the vulnerable call was a single constructor away from the safe one, sitting in plain sight. This suggests static detection is highly tractable: flag `jinja2.Template(`, `jinja2.Environment(`, and `from_string` on non-sandbox environments wherever the template argument is not a compile-time constant. We sketch such a check in §7.

7 Defensive Guidance

For **framework authors**: (1) Never render a non-constant template string with an unsandboxed environment—treat `jinja2.Template(` and `jinja2.Environment(` on variable input as defects, enforceable by a ten-line AST lint. A sandboxed environment is mitigation, not endorsement—Jinja2's own documentation warns against rendering untrusted templates even in the sandbox [13]—so prefer allowlisting the template source (item 3) wherever the template is attacker-supplied. (2) Centralize rendering in one hardened helper (the HF transformers pattern); do not let endpoints grow their own `_get_jinja_env`. Haystack's `HaystackSandboxedEnvironment` [15] is an exemplar of going further: on top of the stock sandbox it refuses attribute access on module objects and refuses calls to callables rooted in dangerous stdlib modules (`os`, `subprocess`, ...) as defense-in-depth. (3) Separate template *selection* from rendering: resolve and allowlist which template may render before compiling it (the vLLM `resolve_chat_template_kwargs` pattern). (4) Bound render resources—timeouts and output caps—because sandboxes do not stop algorithmic-complexity DoS.

For **operators**: (1) Treat model files as untrusted code for the render path: only load chat templates from signed or pinned models, and prefer servers that sandbox rendering (fixed SGLang, current HF transformers/vLLM). (2) Restrict who may author flow configurations that reach template engines (the T2 boundary); audit existing flows for {{/{`chat_template_kwargs` to untrusted clients unless the server version contains the kwargs-filtering fix. (4) Monitor for repeat findings in the same component—a second template CVE in one product is evidence the boundary, not the bug, needs redesign.

8 Related Work

SSTI as a web vulnerability is documented by PortSwigger [11] and practitioner references [12]; these are methodologies, not measurement studies, and do not address AI infrastructure. The "Llama Drama" incident (CVE-2024-34359) was analyzed by Checkmarx [7] and JFrog [8] as a supply-chain case study; our work generalizes the pattern across seven incidents and six frameworks. Prior template-sandbox research focuses on *bypassing* sandboxes (e.g., novel gadget chains) [16, 17]; we show the empirical reality in AI stacks is simpler—most incidents involve no sandbox at all, and our bypass-variant tests confirm current Jinja2 sandboxes hold against documented gadgets. The OWASP AISVS draft [14] independently cites vLLM's CVE-2025-61620 for the point that sandboxed rendering still needs resource limits, converging with our finding F3. Concurrent work by Fogel et al. [18] treats chat templates as executable Jinja2 programs and demonstrates inference-time backdoors implanted via malicious templates across eighteen models and agentic deployments; that work targets supply-chain template *authorship*, whereas we measure framework *rendering* posture across trust boundaries. To our knowledge, no prior work proposes a provenance-based taxonomy of template injection or performs differential sandbox-posture testing across agent frameworks.

[5] https://github.com/infiniflow/ragflow/pull/13305

[6] https://github.com/infiniflow/ragflow/pull/20289: "refactor: remove python source code, part2," merged 2026-09-28, +0/–205285 lines.

[7] https://github.com/vllm-project/vllm/pull/25794: "Add filtering for chat template kwargs," merged 2025-09-27.

9 Conclusion

Prompt-template sandbox escapes are not isolated bugs but a systematic weakness class in AI/agent infrastructure, recurring wherever a template compiler sits on a trust boundary. Our provenance × posture taxonomy places seven CVEs, our differential demonstration (80 cells) confirms the posture column determines exploitability, and version-history analysis shows the same mistake re-implemented across projects and re-appearing in the same product within months. The remediations are small and well-understood—sandbox by default, centralize rendering, allowlist template selection, bound resources—but they must be applied at the boundary, not per bug.

A Ethics

This research involved no human subjects and no live systems. All payload executions ran locally in an isolated VM against pinned historical source versions; RCE-class payloads were read-only (reading a local canary file), causing no writes, no network activity, and no data exfiltration. We used no third-party exploit code, only documented SSTI gadgets reproduced from public methodology references. Our responsible-disclosure sweep of current framework code is a published, re-runnable procedure, not a handwave: `code/sweep_scan.py` is an AST scanner run over pinned repository clones (`repos/`: RAGFlow @ `2400ca8`, SGLang @ `f385390` (2026-10-05), LangChain @ `007cc15` (2026-10-04), LangGraph @ `9a0394d` (2026-10-03), vLLM @ `0c16eee` (2026-10-05)) plus the vulnerable tags RAGFlow `v0.24.0` and SGLang `v0.5.9` as positive controls. It flags every `jinja2.Template(`, `jinja2.Environment(`, and `.from_string(` call site whose file shows no sandbox hint; flagged sites are then triaged by hand. Run 2026-10-09 (`logs/sweep_scan.log`): the vulnerable tags flag the known CVE sites (confirming the scanner sees them); at HEAD the only flags are test-only code, build-time CUDA kernel codegen (vLLM `csrc/`, constant templates rendered at build time, not at serving time), and one constant-template call in SGLang's `tiktoken_tokenizer.py` (template is a compile-time constant two lines above the call). HF transformers HEAD was inspected directly (`chat_template_utils.py:495` uses `ImmutableSandboxedEnvironment`, verified at commit `541cf67a`, 2026-09-30). No new live vulnerabilities were found, so no vendor disclosure was required; the disclosure log is included with our artifacts. We disclose that the vulnerable versions tested correspond to already-published CVEs with vendor patches available, and our results on fixed versions serve as independent fix verification. We believe publishing the taxonomy and defensive guidance reduces net risk by helping framework authors eliminate the weakness class.

B Open Science

Artifacts: (1) the 10-payload corpus (`corpus/escape_payloads.py`); (2) the differential-testing harness (`code/full_harness.py`, `code/pilot_harness.py`) with pinned source references; (3) the vLLM kwargs-override logic replay (`code/vllm_kwargs_override_demo.py`); (4) raw results JSON and rerun logs (`results/`, `logs/`); (5) the disclosure log (`DISCLOSURE_LOG.md`); (6) this paper's LaTeX source. An anonymous artifact bundle is provided for review via anonymous.4open.science at https://anonymous.4open.science/r/template-provenance-sandbox-escapes-7C4D/; on acceptance, artifacts will be published at a stable public URL. Reproduction requires only Python 3.12 and `jinja2` (CPU-only, no API keys, no network beyond cloning the cited public repositories at the pinned tags/commits).

C AI Use Disclosure

The author used the AI tool Muse (Muse Spark 1.3, Meta) for drafting, literature search, editing, and code scaffolding. The contents are reviewed, edited, and attested for accuracy by the author. All experimental results were produced by the deterministic harnesses in the artifact bundle and were independently re-run; all quantitative claims were recomputed from raw outputs; all cited sources were verified against their primary publications by the author. The author reviewed all AI-assisted content as if self-written and takes full responsibility for its accuracy, originality, and integrity, per the USENIX SAIS '27 AI-use policy.

D Corpus and Version Pins

Payloads P1–P6, B1–B4 are listed with sources in `corpus/escape_payloads.py`. All harness runs used Jinja2 3.1.6; the sandbox-block outcomes are contingent on that version's sandbox behavior. Render outcomes are deterministic (one run per cell is sufficient). Executed render-posture cores (A–H, Table 2) use pinned vulnerable versions and current HEAD: RAGFlow `v0.24.0` (vulnerable) and post-#13305 (merged 2026-03-02); SGLang `v0.5.9` (vulnerable) and HEAD post-#23660 (commit `f385390`, 2026-10-05); LangChain HEAD (commit `007cc15`, 2026-10-04, `libs/core/langchain_core/prompts/string.py:72`); llama-cpp-python $<$0.2.72 (advisory-mapped posture, GHSA-56xg-wfcc-g829); Giskard giskard-agents $\leq$0.3.3 (advisory-mapped posture, GHSA-frv4-x25r-588m); HF transformers HEAD (`src/transformers/utils/chat_template_utils.py:495`, verified at commit `541cf67a`, 2026-09-30). Inspected but not executed: LangGraph HEAD (commit `9a0394d`,

2026-10-03; code search over `libs/` finds no Jinja2 usage—negative control by inspection, not a harness path); vLLM HEAD (commit `0c16eee`, 2026-10-05, `vllm/renderers/hf.py`—only the kwargs-override *logic* is replayed in `code/vllm_kwargs_override_demo.py`, not a render path); RAGFlow HEAD (commit `2400ca8`; Go agent rewrite, no Jinja2 render surface, confirmed by the sweep in Appendix A).

Data and Code Availability

The audit code, sweep corpus records, and reproduction scripts are archived in the anonymized artifact at https://anonymous.4open.science/r/template-provenance-sandbox-escapes-7C4D/.

Competing Interests

The author declares no competing interests. No funding was received for this work.

References

[1] GitHub Security Advisory GHSA-vvwj-fvwh-4whx. Server-side template injection leading to RCE in RAGFlow agent "Text Processing" component (CVE-2026-28797), Apr. 2026.

[2] GitLab Security Advisory, Oct. 2, 2026. Prompt template sandbox escape in self-hosted AI Gateway (CVE-2026-90970), CVSS 9.9; fixed in 19.2.4/19.3.2/19.4.1.

[3] GitLab Security Advisory, Feb. 6, 2026. Insecure template expansion in Duo Workflow Service via crafted flow definitions (CVE-2026-1868), CVSS 9.9; fixed in 18.6.2/18.7.1/18.8.1.

[4] SGLang fix PR #23660, Apr. 29, 2026. Unsandboxed Jinja2 chat-template rendering in `/v1/rerank` via malicious GGUF `tokenizer.chat_template` (CVE-2026-5760), CVSS 9.8.

[5] GitHub Security Advisory GHSA-frv4-x25r-588m. Server-side template injection via `ChatWorkflow.chat()` using non-sandboxed Jinja2 `Environment` in Giskard Agents (CVE-2026-34172), Mar. 2026; CVSS 8.8 (v3.1); fixed in 0.3.4/1.0.2b1. https://github.com/advisories/GHSA-frv4-x25r-588m

[6] GitHub Security Advisory GHSA-56xg-wfcc-g829 ("Llama Drama"). Jinja2 SSTI via model-file chat template in llama-cpp-python (CVE-2024-34359), CVSS 9.7; fixed in 0.2.72.

[7] Checkmarx, May 2024. "Llama Drama": researchers uncover critical flaw in llama-cpp-python.

[8] JFrog Research, model-threats/GGUF-SSTI.md. GGUF model files as SSTI vectors.

[9] NIST National Vulnerability Database. CVE-2025-61620: vLLM OpenAI-compatible server resource exhaustion via `chat_template` parameters, CVSS 3.1 6.5, CWE-400. https://nvd.nist.gov/vuln/detail/CVE-2025-61620

[10] Miggo Research. CVE-2025-61620: vLLM OpenAI server Jinja DoS — write-up of the `chat_template` parameter handling flaw. https://www.miggo.io/vulnerability-database/cve/CVE-2025-61620

[11] PortSwigger Web Security Academy. Server-side template injection methodology.

[12] The Hacker Recipes. SSTI: theory and practice.

[13] Pallets Projects. Jinja2 sandbox documentation (jinja.palletsprojects.com).

[14] OWASP AISVS 1.0, C04-01 Workload Sandboxing Validation (draft). Cites CVE-2025-61620 for sandbox-insufficient resource limits.

[15] deepset-ai/haystack, `haystack/utils/jinja2_sandbox.py` (HEAD). `HaystackSandboxedEnvironment`: hardened Jinja2 sandbox refusing module attribute access and calls into dangerous stdlib modules.

[16] Rémi GASCOU (Podalirius), "Optimizing Server Side Template Injection payloads for Jinja2," GreHack 2021. https://podalirius.net/en/publications/grehack-2021-optimizing-ssti-payloads-for-jinja2/GreHack_2021_-_Optimizing_Server_Side_Template_Injections_payloads_for_jinja2_paper.pdf

[17] SecureLayer7, "What is Jinja2 template injection?," application-security reference. http://securelayer7.net/learn-pdf/application-security/jinja2-template-injection.pdf

[18] Ariel Fogel, Omer Hofman, Eilon Cohen, Roman Vainshtein, "Inference-Time Backdoors via Chat Templates: From LLM Supply Chains to Agentic System Compromise," arXiv:2602.04653 [cs.CR], May 2026. https://arxiv.org/abs/2602.04653